

Article

GALR: Graph-Based Root Cause Localization and LLM-Assisted Recovery for Microservice Systems

Wenya Zhang ¹, Zhi Yang ¹, Fang Peng ¹, Le Zhang ¹, Yiting Chen ¹ and Ruibo Chen ^{2,*}

¹ The Information & Telecommunication Center of STATE GRID Corporation of China, Beijing 100033, China; wenya-zhang@sgcc.com.cn (W.Z.); zhi-yang@sgcc.com.cn (Z.Y.); fang-peng@sgcc.com.cn (F.P.); le-zhang@sgcc.com.cn (L.Z.); yiting1-chen@sgcc.com.cn (Y.C.)

² School of Computer Science and Engineering, Beihang University, Beijing 100191, China

* Correspondence: crbemail@163.com

Abstract

With the rapid evolution of cloud-native platforms, microservice-based systems have become increasingly large-scale and complex, making fast and accurate root cause localization and recovery a critical challenge. Runtime signals in such systems are inherently multimodal—combining metrics, logs, and traces—and are intertwined through deep, dynamic service dependencies, which often leads to noisy alerts, ambiguous fault propagation paths, and brittle, manually curated recovery playbooks. To address these issues, we propose GALR, a graph- and LLM-based framework for root cause localization and recovery in microservice-based business middle platforms. GALR first constructs a multimodal service call graph by fusing time-series metrics, structured logs, and trace-derived topology, and employs a GAT-based root cause analysis module with temporal-aware edge attention to model failure propagation. On top of this, an LLM-based node enhancement mechanism infers anomaly, normal, and uncertainty scores from log contexts and injects them into node representations and attention bias terms, improving robustness under noisy or incomplete signals. Finally, GALR integrates a retrieval-augmented LLM agent that retrieves similar historical cases and generates executable recovery strategies, with consistency checking against expert-standard playbooks to ensure safety and reproducibility. Extensive experiments on three representative microservice datasets demonstrate that GALR consistently achieves superior Top-*k* accuracy and mean reciprocal rank for root cause localization, while the retrieval-augmented agent yields substantially more accurate and actionable recovery plans compared with graph-only and LLM-only baselines, providing a practical closed-loop solution from anomaly perception to recovery execution.

Keywords: microservice systems; root cause localization; graph attention networks; large language models



Academic Editor: Mehdi Sookhak

Received: 14 November 2025

Revised: 26 December 2025

Accepted: 29 December 2025

Published: 5 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

With the rapid evolution of cloud-native technologies and the increasing complexity of large-scale online applications, microservice architecture has become a prevailing paradigm for modern distributed systems [1]. By decomposing monolithic applications into loosely coupled services, microservices improve agility and scalability, but their high dynamism and deep interdependence also increase operational fragility: minor local failures may propagate along dependency chains and trigger system-level degradation. Therefore, adaptive fault management—detecting anomalies, localizing root causes, and generating

effective recovery strategies with minimal human intervention—remains a critical challenge for reliability assurance in microservice systems.

Recent advances in AIOps and graph-based learning have improved automated fault diagnosis and propagation modeling. Graph neural networks (GNNs) enable relational reasoning over service dependencies, while multimodal observability data (logs, metrics, and traces) improve diagnostic coverage and interpretability in practice [2–7]. Building on these foundations, a range of root cause analysis (RCA) methods have been developed to capture failure propagation and pinpoint faulty services. Nevertheless, practical self-healing in microservice systems is still limited by several gaps.

First, diagnosis and recovery are often treated as separate stages: localization results are not always expressed in a form that can be directly consumed by downstream recovery decision-making, and feedback from recovery outcomes is rarely incorporated into subsequent diagnosis. Second, although multimodal fusion is widely studied, unstructured logs are commonly reduced to coarse templates or sparse categories, which limits how log semantics are aligned with dependency-aware graph reasoning. Third, recovery policies in many systems still rely on static heuristics or hand-crafted playbooks, which may not adapt well to evolving dependencies, workload drift, or fault combinations.

Large language models (LLMs) provide an alternative way to interpret unstructured logs and synthesize recovery suggestions, and recent tool-augmented paradigms further enable iterative planning in structured environments [8–12]. However, applying LLMs to microservice AIOps remains challenging because LLMs do not natively operate over graph-structured dependencies and quantitative telemetry, and their free-form outputs require grounding and verification to be safe and reproducible. Existing hybrid GNN–LLM efforts often adopt a loose coupling between graph inference and natural-language recommendation, leaving open how to inject LLM-derived semantics into graph reasoning while keeping recovery decisions verifiable.

To address these challenges, we propose GALR, a unified framework for root cause localization and recovery in microservice systems. GALR constructs a multimodal service call graph from aligned metrics, logs, and traces. For localization, it adopts a GAT-based module that incorporates temporal decay on edges and LLM-derived anomaly probability triplets at nodes and further introduces a semantic bias to guide attention toward high-risk regions. For recovery, a retrieval-augmented LLM agent retrieves similar historical incidents and generates recovery plans, whose action sets can be quantitatively compared against expert-standard playbooks to support verification and evaluation. In this way, GALR forms an interpretable loop from anomaly perception to localization and strategy generation, while remaining compatible with existing monitoring and incident management workflows.

The main contributions of this paper are summarized as follows:

- We propose GALR, a unified framework that couples graph neural networks with large language models to support root cause localization and recovery planning for microservice systems.
- We design a GAT-based localization module on a multimodal service call graph, where LLM-derived anomaly probability triplets and a semantic bias are integrated with temporal decay factors to enhance attention-based propagation modeling.
- We develop a retrieval-augmented recovery agent that generates recovery strategies conditioned on localized root causes and retrieved historical incidents, and evaluates generated plans via quantitative matching against standard playbooks.
- We conduct experiments on three representative microservice datasets (Customer Service, Power Grid Resource, and SockShop), comparing GALR with GNN baselines

and recent RCA methods, and further analyze the effects of LLM-based semantic enhancement and attention design via ablation studies.

The remainder of this paper is organized as follows. We first review representative studies on microservice diagnosis and recovery to position our work within graph-based, causal, multimodal, and closed-loop paradigms. We then present GALR and detail multimodal data processing, graph-based localization, and LLM-assisted semantic enhancement and recovery planning. Next, we describe the datasets, evaluation protocol, and implementation settings, followed by experimental results and ablation analyses. Finally, we discuss key findings and limitations, and conclude the paper.

2. Related Work

Research on diagnosis and recovery for microservice systems has evolved from correlation-based anomaly scoring on single signals to graph-centric, multimodal, and increasingly causal formulations, driven by industrial demands for interpretability, on-line adaptability, and auditable decision making [7]. Early experiences in production emphasized the difficulties of noisy and partial observability, fast-changing deployments, and cross-layer dependencies, which in turn motivated methods that reason jointly over topology and telemetry rather than over isolated metrics [7]. This evolution produced three complementary strands: (i) graph-based root cause analysis (RCA) that propagates abnormality along dependency structures; (ii) causal approaches that learn directionality and intervention effects; and (iii) multimodal, hierarchical observability that fuses logs, metrics, and traces across granularities. Recent work further extends these strands with learning-based controllers and large language models (LLMs) to translate diagnostic evidence into actionable remediation [13–16]. GALR follows the graph-centric and multimodal lines for dependency-aware localization, and differs from loosely coupled pipelines by injecting LLM-derived semantic priors into graph attention and using retrieval-augmented recovery planning with quantitative consistency checking to improve the actionability and verifiability of recovery suggestions.

2.1. Graph-Centric Root Cause Analysis in Microservices

Graph-centric RCA has established itself as a robust and scalable paradigm for large microservice deployments. Methods in this family construct a service-call or resource-dependency graph and propagate anomaly evidence via random walks, ranking, or spectral mechanisms to localize suspects with minimal supervision [17–20]. To reduce instrumentation assumptions and cope with black-box environments, topology-agnostic techniques such as CloudRanger and FChain infer influence flows without complete tracing coverage [21,22]. At the cluster edge, FluxRank targets machine-level localization to complement service-level reasoning [23]. A parallel line improves robustness to partial or coarse-grained observability by performing end-to-end reasoning directly on traces and performance spectra (Microscope, Automap, Microrank, MicroSketch, TraceRank), thereby handling missing spans, sampling, and heterogeneous instrumentation [24–28]. These results collectively validate the hypothesis that structural priors and propagation-aware scoring are effective at cloud scale. However, many graph-centric RCA methods remain correlation-driven and often incorporate log information only through coarse templates or sparse features, which limits their ability to leverage rich log semantics in complex incidents. GALR builds on dependency-aware propagation, while introducing LLM-derived anomaly probability triplets and a semantic bias in attention to better align log semantics with graph reasoning during localization.

2.2. Causal Diagnosis and Intervention Modeling

While propagation-based methods are powerful, their reliance on correlation can be brittle in the presence of confounders, concurrent incidents, and multi-hop effects. Causal formulations address these pitfalls by learning directed relations and modeling intervention effects. Interdependent Causal Networks (ICN) explicitly represent cross-subsystem dependencies and report improved localization stability, while CORAL advances incremental causal graph learning, reducing retraining costs and enabling online adaptation as services evolve [29,30]. In domain-focused settings—e.g., OLTP databases—CauseRank leverages structural invariants for generalizable diagnosis, and MicroCause demonstrates causality-driven localization in microservice scenarios with promising precision and interpretability [31,32]. Despite their strengths, causal learners may require stable assumptions and sufficient data for causal discovery, which can be challenging under fast-changing deployments and heterogeneous instrumentation. GALR is compatible with such causal signals when available, but does not assume full causal identifiability; instead, it focuses on a practical graph substrate that can incorporate causal edges or constraints without changing the overall pipeline.

2.3. Multimodal Observability and Log-Centric Reasoning

Production-grade RCA also hinges on fusing heterogeneous evidence and reasoning across granularities. End-to-end troubleshooting frameworks such as EADRO unify multiple telemetry sources (logs, metrics, traces) to improve coverage under real workloads [33]. TrinityRCL pushes granularity to the code level by aligning multi-type signals across the service–interface–request–code hierarchy, enabling developers to connect system symptoms to actionable artifacts [34]. For unstructured and interleaved logs—long recognized as a major operational pain point—LogKG injects knowledge-graph priors to improve inference over noisy narratives, whereas SwissLog prioritizes robustness to real-world log variability [35,36]. Complementary efforts mine exception logs and hybrid log–graph structures to increase recall and reduce false negatives in complex failure modes [37–39]. Although multimodal fusion improves evidence coverage, many systems still treat logs as auxiliary signals and the alignment between log semantics and dependency structures is not fully addressed, especially when logs are reduced to templates or shallow features. GALR follows the evidence-unification principle, but explicitly injects LLM-derived semantic priors into node representations and attention computation to narrow the semantic gap of logs during graph-based localization.

2.4. Towards Closed-Loop and Interpretable Recovery

A parallel set of advances aims to close the loop from localization to automated recovery. Hierarchical reinforcement learning with human feedback (HRLHF) has been applied to microservice operations, showing that policy hierarchies can capture operator preferences and safety constraints while learning effective mitigation sequences [40]. InstantOps further couples failure prediction with RCA to tighten response windows and support proactive interventions [41]. In tandem, LLM-assisted incident analysis has emerged as a practical vehicle for converting semi-structured tickets, playbooks, and historical cases into hypotheses and remediation recommendations. Studies demonstrate LLM agents that coordinate RCA steps, LLM-based mitigation recommenders trained on operational knowledge, and in-context learning pipelines tailored to incident narratives and telemetry snippets [13–16]. However, these LLM systems typically lack strong structural grounding, risking hallucinated or unsafe suggestions. This motivates GALR to treat the graph substrate and operational references as constraints for recovery planning, and to evalu-

ate generated actions through quantitative consistency checking rather than relying on free-form recommendations alone.

Finally, empirical and industrial studies continue to articulate desiderata that shape GALR's design: interpretability and auditability across layers, resilience to drift and evolving dependencies, and the ability to relate system symptoms to actionable engineering artifacts [7]. Deep-learning-based diagnosis in cloud microservices confirms the utility of learned representations but also exposes generalization challenges in online settings and under changing workloads [42]. Change-aware localization (e.g., ChangeRCA) exemplifies the benefits of aligning diagnostic evidence with code and configuration changes that developers can act upon [43]. In response, GALR follows these requirements by integrating dependency-aware graph reasoning with multimodal evidence and an LLM-assisted recovery module that is explicitly constrained and evaluated.

In summary, prior studies on microservice diagnosis and recovery can be grouped into four lines: graph-centric RCA, causal diagnosis, multimodal troubleshooting, and closed-loop or LLM-assisted remediation. Graph-centric methods scale well by propagating anomaly evidence along dependency structures, but they are often correlation-driven and tend to under-utilize unstructured logs beyond coarse templates. Causal approaches improve robustness to confounders and concurrent incidents, yet their deployment typically depends on stable assumptions and sufficient data for causal discovery. Multimodal frameworks enhance coverage by combining logs, metrics, and traces, but many pipelines still separate detection, localization, and recovery, leaving localization outputs not directly actionable for downstream operations. Recent LLM-assisted systems are promising for producing interpretable hypotheses and remediation suggestions, but their recommendations can be weakly grounded in system topology and are rarely equipped with explicit verification against operational constraints.

These limitations motivate the design choices in GALR. We build on the dependency-aware reasoning of graph-based RCA and the evidence coverage of multimodal observability, while injecting LLM-derived anomaly probability triplets into graph attention to reduce the semantic gap of logs during localization. For recovery, we adopt retrieval-augmented planning and evaluate generated actions by quantitative consistency checking against standard playbooks, aiming to improve the reproducibility and operational trustworthiness of suggested mitigation steps.

3. Methodology

This paper proposes an intelligent self-healing analysis and decision-making framework for microservice-based business middle platforms. Our objective is to support an end-to-end workflow from anomaly perception to root cause localization and recovery suggestion, by jointly leveraging multimodal observability data and service dependency structures. Given windowed metrics, logs, and traces collected from a microservice system, we construct a service call graph and formulate root cause localization as a node-level identification problem under practical class imbalance. The proposed framework integrates four core modules: data processing and multimodal fusion, GNN-based root cause localization, LLM-based node feature enhancement, and case retrieval and generation based recovery strategy planning. As illustrated in Figure 1, these modules form a coherent pipeline from multimodal evidence alignment to localization evidence and strategy generation.

To improve readability and avoid ambiguity in the following formulation, Table 1 summarizes the main symbols used in our methodology, including the service call graph, multimodal node features, attention-related terms, and the notations for retrieval-based recovery planning.

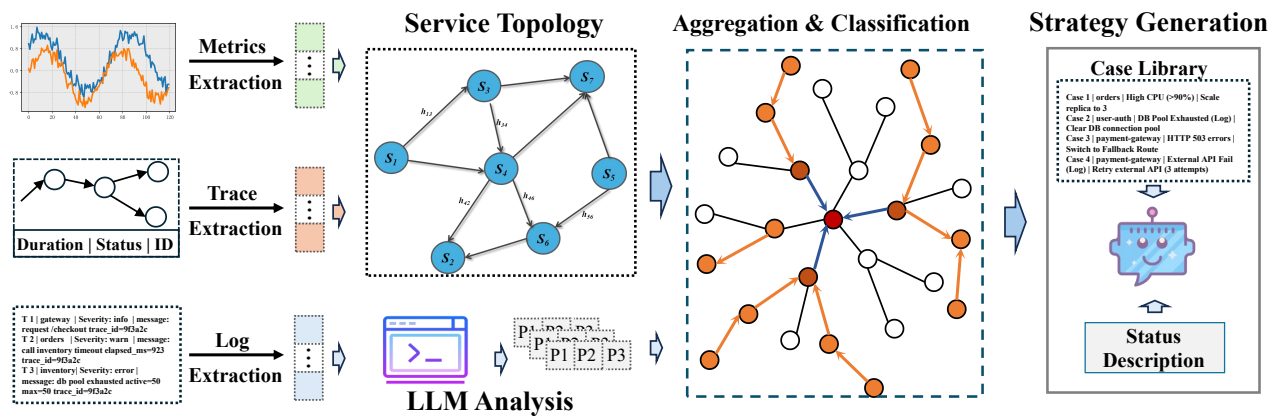


Figure 1. Overview of the proposed GALR framework for intelligent root cause localization and recovery in microservice systems.

Table 1. Main notations used in the methodology.

Symbol	Meaning
G	Service call graph constructed from traces
V	Node set in the service call graph
E	Directed edge set in the service call graph
$\mathcal{N}(i)$	Neighbor set of node i used for aggregation
x_i	Metric feature vector of node i within a time window
e_i	Log semantic embedding of node i
h_i^{base}	Base multimodal node representation
$h_i^{(l)}$	Node feature of i at GAT layer l
$\alpha_{ij}^{(l)}$	Attention weight from neighbor j to node i at layer l
ω_{ij}	Temporal decay factor associated with edge (i, j)
Δt_{ij}	Time gap between adjacent invocation events on edge (i, j)
r_i	Predicted probability that node i is a root cause
y_i	Ground-truth root-cause label of node i
s_i	LLM-derived probability triplet for node state
Δ_{ij}	Semantic bias term injected into attention computation
$\mathcal{C}$	Recovery case library
Sim_{cos}	Cosine similarity used for case retrieval
Correctness	Action-overlap metric for evaluating generated recovery plans

3.1. Data Processing and Multimodal Fusion

In a business middle platform architecture, a microservice system is composed of a large number of mutually dependent service components. The runtime state of the system can be comprehensively characterized by multi-source monitoring data, including performance metrics, logs, and traces. These data sources differ significantly in sampling frequency, semantic granularity, and structural form. Therefore, before model training, unified structuring, temporal alignment, and feature normalization are required to construct multimodal feature representations that can be jointly leveraged by subsequent graph models and LLMs.

3.1.1. Preprocessing of Time-Series Metrics and Feature Construction

In the preprocessing stage of metrics, the goal is to transform raw, high-dimensional monitoring records into unified and normalized feature vectors, so as to eliminate heterogeneity across different metrics and improve model efficiency. Specifically, for each service or system component, we first consider its time-series metric observations (such as response latency, CPU utilization, QPS, memory usage, etc.). Within a given time window,

heterogeneous metric records associated with the same service entity are aggregated to form a unified metric vector denoted by X_i . A sliding window mechanism is then applied along the temporal dimension to segment long sequences into a series of local time intervals, and a feature vector is constructed for each window to capture local dynamic behavior.

To eliminate the influence of different scales and value ranges, a hybrid normalization strategy is adopted. For most continuous metrics (such as response latency and CPU utilization), Z-score normalization is applied so that the data have zero mean and unit variance at the global level:

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}, \quad (1)$$

where $\tilde{x}_{i,j}$ is the normalized value, $x_{i,j}$ is the original observation, and μ_j and σ_j denote the mean and standard deviation of metric j over the observation period, respectively. For resource or performance metrics with bounded ranges where relative amplitude is of primary interest, min–max normalization is used:

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \min(x_j)}{\max(x_j) - \min(x_j)}, \quad (2)$$

which highlights the relative variation of a metric within its own value range.

For discrete metrics such as error counts or restart times, appropriate numerical representations are constructed according to their business semantics and distributional properties, e.g., through frequency mapping or one-hot encoding. To reduce redundancy and enhance interpretability, all raw metrics are grouped into several categories based on their semantics and business logic, including resource, network, performance, system behavior, and availability/stability metrics. Category-specific normalization strategies as described above are then applied. Subsequently, for each sliding window, statistical features such as mean, variance, slope, kurtosis, maximum, and minimum are extracted from the normalized sequences to form the service feature vector $x_i \in \mathbb{R}^{d_x}$ within that time window, which serves as the basic input to the subsequent GNN and fusion models.

3.1.2. Log Structuring and Semantic Feature Extraction

Log data are typically long, unstructured text, which, if directly used for modeling, would lead to high dimensionality, substantial noise, and ambiguous semantics. To address this issue, a two-stage progressive parsing strategy is adopted to transform raw logs into structured, semantically focused tabular data, thereby providing a foundation for multi-modal fusion and node feature construction. In the outer parsing stage, the outermost JSON structure is parsed to extract the core log content string `log`, while key meta-information such as `stream` and `severity` is recorded simultaneously. In the inner parsing stage, if the `log` field can be further parsed as a nested JSON structure, internal structured fields are extracted, with particular emphasis on the message key that carries actual business semantics. The prefix key path is concatenated to form a `log_type` label, which is used to distinguish logs from different sources or of different types.

After structuring, regular expressions are applied to the message field to extract key identifiers. The corresponding text segment is then cropped and normalized as the core semantic unit `log_message`, thereby filtering out redundant prefixes and suffixes that are weakly related to root cause analysis. In our current implementation, we use `log_type` as the log-side feature signal for each service node. Specifically, we build a fixed vocabulary of `log_type` values on the training split and map each `log_type` to an index, which is then encoded by a trainable embedding table. For a given node i and time window, we collect all `log_type` tokens observed within the window, look up their embeddings, and apply mean pooling to obtain a window-level log representation $e_i \in \mathbb{R}^{d_e}$. Finally, e_i is temporally

aligned with the corresponding metric feature vector x_i within the same window, enabling consistent multimodal fusion for subsequent graph modeling and node enhancement.

3.1.3. Trace Parsing and Graph Topology Construction

Trace data capture the propagation paths of requests and the dependency structure among services in a microservice system, and thus constitute the core basis for multimodal fusion and graph structural modeling in this work. In practical deployments, a user request typically passes through multiple microservices sequentially, where each inter-service invocation is recorded as a span. Spans are connected via parent–child relationships to form complete traces, which describe the request path, inter-service dependencies, stage-wise latency, and invocation outcomes.

In the preprocessing stage, raw trace data are first parsed and normalized to extract key attributes of each span, including span ID, service name, operation name, start time, duration, and parent span ID. These attributes define the call sequence and dependency relations among services. Using the trace structure as the backbone, service instances or spans are abstracted as the node set V of a graph, and parent–child invocations are abstracted as the directed edge set E , thereby constructing a weighted directed graph $G = (V, E)$ as the structural input for the subsequent GNN.

For node feature construction, multimodal features from metrics and logs are fused at each node i . Specifically, the log semantic feature e_i is concatenated with the time-series metric feature x_i to form the basic node representation

$$h_i^{\text{base}} = [x_i \| e_i], \quad (3)$$

where $\|$ denotes vector concatenation. In this way, the graph topology not only encodes the static dependency structure among services but also carries the dynamic runtime state and potential anomaly patterns within each time window. This unified structural representation supports downstream GNN-based fault propagation analysis and root cause localization.

3.2. GNN-Based Root Cause Localization

For root cause localization, the service call graph $G = (V, E)$ with fused multimodal features is used as input, and a Graph Attention Network (GAT) is employed to perform node-level root cause identification. By learning adaptive attention weights between services, the model assigns higher importance to critical propagation paths and potential root cause nodes, thereby enabling fine-grained characterization of fault propagation.

In the l -th GAT layer, the feature $h_i^{(l)}$ of node i is updated via neighborhood aggregation. The update rule is given by

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)} \right), \quad (4)$$

where $\mathcal{N}(i)$ denotes the neighbor set of node i , $W^{(l)}$ is the feature transformation matrix in layer l , $\sigma(\cdot)$ is a nonlinear activation function (e.g., ReLU), and $\alpha_{ij}^{(l)}$ is the attention weight assigned to neighbor j when aggregating information for node i , reflecting the strength of information propagation between them. To better integrate the semantic priors provided by the LLM, a learnable semantic bias term is introduced into the attention computation, whose specific form will be detailed in the next subsection.

Considering that service interactions in traces exhibit explicit temporal ordering and causal delay, a temporal decay factor ω_{ij} is introduced into edge features to model temporal consistency constraints across services:

$$\omega_{ij} = \exp\left(-\frac{\Delta t_{ij}}{\tau}\right), \quad (5)$$

where Δt_{ij} denotes the time difference between adjacent invocation events, and τ is a time constant. This temporal decay factor serves as a structural prior to adjust the attention weights, yielding the reweighted attention coefficient

$$\alpha'_{ij} = \omega_{ij} \cdot \alpha_{ij}^{(l)}, \quad (6)$$

so that temporally closer invocations with potential causal relations receive higher weights during aggregation. The model finally outputs the probability r_i of each node being a root cause, and is trained using a binary cross-entropy loss:

$$\mathcal{L}_{\text{rca}} = - \sum_{i \in V} [y_i \log r_i + (1 - y_i) \log(1 - r_i)], \quad (7)$$

where $y_i \in \{0, 1\}$ indicates whether node i is a true root cause. With this design, GAT jointly exploits multimodal features and call topology to automatically focus on critical nodes and paths that are more likely to trigger anomalies, thereby achieving accurate root cause localization.

3.3. LLM-Based Node Feature Enhancement

Due to the highly unstructured and semantically complex nature of raw logs, traditional pattern-matching or shallow-feature-based methods struggle to fully exploit the latent anomaly patterns they contain. To address this challenge, an LLM is introduced to perform semantic reasoning and feature enhancement on structured log content, and to generate a compact three-dimensional probability label s_i for each service node v_i in the call graph, thereby injecting deep log semantics into the graph model in a learnable form.

Concretely, for each node i , an LLM-based inference is performed on its log context to obtain a three-dimensional probability vector

$$s_i = [p_{\text{anom}}, p_{\text{norm}}, p_{\text{uncert}}], \quad (8)$$

where p_{anom} denotes the confidence that the node is in an anomalous state, p_{norm} is the confidence for a normal state, and p_{uncert} reflects the degree of uncertainty, with the three components summing to 1. This soft-label representation preserves the uncertainty and vagueness inherent in log semantics, while avoiding hard misclassification that may arise from binary labels. Finally, the semantic label is concatenated with the original metric and log embedding features to form the enhanced node representation:

$$h_i = [x_i || e_i || s_i] \in \mathbb{R}^{d_x + d_e + 3}, \quad (9)$$

where x_i is the metric feature, e_i is the log semantic embedding, and s_i is the semantic prior from the LLM, thus achieving unified multimodal fusion at the node representation level.

To effectively drive the LLM to output the above probability labels, a set of prompt templates is designed. The prompt specifies the role of the model (e.g., “log analysis assistant”) and instructs it to determine the current service state based on the given ServiceName and log content, and to output a JSON structure with three fields: `anomaly_prob`, `normal_prob`, and `uncertainty`, constrained to sum to 1, along with an example output. For specific services, service-specific rules can be explicitly embedded into the prompt, e.g., “if “Start charge card” appears but “Charge successfully” is missing, treat it as a potential anomaly”, thereby injecting business priors into the LLM’s reasoning process.

After obtaining node-level anomaly confidences, the semantic prior is further integrated into the graph attention mechanism to emphasize high-risk paths. A semantic bias term Δ_{ij} is introduced:

$$\Delta_{ij} = \lambda \cdot p_i^{\text{anom}} \cdot p_j^{\text{anom}}, \quad (10)$$

where p_i^{anom} and p_j^{anom} are the anomaly probabilities of node i and its neighbor j , respectively, and λ is a learnable weight. The modified attention score is then given by

$$e'_{ij} = e_{ij} + \Delta_{ij}, \quad (11)$$

which is subsequently normalized via a Softmax function to obtain the final attention weights used for aggregation. By introducing this semantic bias, the GNN tends to propagate information along service call paths with higher anomaly confidence, thus improving its ability to identify critical fault propagation paths and root cause nodes.

3.4. Case Retrieval and Generation Based Recovery Strategy Planning

After root causes have been localized, the system further generates reasonable and safe recovery strategies for the identified root cause nodes to achieve service-level self-healing. To this end, we design an LLM-based recovery strategy agent that produces executable and verifiable recovery plans through a pipeline of similar case retrieval, strategy generation, and consistency verification.

First, a standard recovery case library $\mathcal{C}$ is constructed to store historical fault scenarios, root cause descriptions, and their corresponding recovery actions. Each case represents a typical fault-handling pattern observed in practice and is associated with a set of expert-defined recovery actions. Given a newly detected root cause cause , both its description and those of the cases in the library are mapped into the same embedding space, denoted by $\hat{h}$ and h_i , respectively. The cosine similarity between the target root cause and each historical case is computed as

$$\text{Sim}(\text{cause}, \text{cause}_i) = \text{Sim}_{\cos}(\hat{h}, h_i). \quad (12)$$

Cases with the highest similarity scores are selected as contextual prompts and combined with current system status information as input to the LLM for generating candidate recovery plans. The prompt typically includes the root cause description, current resource and performance states, and representative recovery actions from retrieved cases, and asks the model to output a structured recovery plan containing action steps, verification metrics, rollback strategies, and expected outcomes, which facilitates subsequent programmatic execution and automated evaluation.

To assess the quality of generated recovery plans, we measure their consistency with standard recovery actions. Let $\text{Actions}_{\text{gen}}$ denote the set of recovery actions in the generated plan and $\text{Actions}_{\text{std}}$ the corresponding action set in the matched standard case. Since the recovery actions considered in our setting correspond to atomic operational procedures commonly used in microservice systems and do not exhibit strict execution-order dependencies, we focus on action-level coverage rather than sequence-level alignment. The consistency score is defined as

$$\text{Correctness} = \frac{|\text{Actions}_{\text{gen}} \cap \text{Actions}_{\text{std}}|}{|\text{Actions}_{\text{std}}|}. \quad (13)$$

When this score exceeds a predefined threshold and no obvious conflicting operations are introduced, the generated strategy is considered to be consistent with expert knowledge and can be further deployed into controlled online validation and canary release stages. Experimental results show that under typical fault scenarios, the recovery strategies gener-

ated by the proposed agent exhibit high agreement with expert-designed solutions, while maintaining good interpretability and transferability across different fault types.

3.5. Overall Procedure and Implementation Notes

Algorithm 1 summarizes the overall pipeline of GALR in a time-windowed manner. For each window, we parse traces to construct the directed service call graph $G = (V, E)$, and align windowed metrics and logs to form node-level multimodal signals. Concretely, metrics within the window are aggregated into x_i , and logs within the same window are instantiated by `log_type` and mean-pooled to obtain e_i . We then query the LLM on the node's log context to obtain the probability triplet $s_i = [p_{\text{anom}}, p_{\text{norm}}, p_{\text{uncert}}]$, and form the initial node feature $h_i = [x_i || e_i || s_i]$. On $(G, \{h_i\})$, the GAT module incorporates the temporal decay ω_{ij} and semantic bias Δ_{ij} to produce root-cause scores $\{r_i\}$. Finally, we select top-K candidates by $\{r_i\}$, retrieve top-k similar cases from $\mathcal{C}$ using Sim_{cos} , and prompt the LLM to generate recovery actions $\text{Actions}_{\text{gen}}$, which are evaluated by Correctness. Additional implementation settings and hyperparameters are reported in Section 4.3.

Algorithm 1 Overall pipeline of GALR

Require: Windowed metrics/logs/traces; case library $\mathcal{C}$

Ensure: Ranked root-cause candidates $\{r_i\}$ and a recovery plan

- 1: Parse traces $\rightarrow G = (V, E)$
 - 2: Parse logs $\rightarrow \text{log_type}$; mean-pool within window $\rightarrow e_i$
 - 3: Aggregate metrics within window $\rightarrow x_i$; align (x_i, e_i) by window timestamp
 - 4: Query LLM on log context $\rightarrow s_i = [p_{\text{anom}}, p_{\text{norm}}, p_{\text{uncert}}]$
 - 5: Form node features $h_i = [x_i || e_i || s_i]$
 - 6: Run GAT with ω_{ij} and Δ_{ij} on $(G, \{h_i\}) \rightarrow \{r_i\}$
 - 7: Select top-K candidates by $\{r_i\}$; retrieve top-k cases from $\mathcal{C}$ by Sim_{cos}
 - 8: Prompt LLM with retrieved cases $\rightarrow \text{Actions}_{\text{gen}}$; evaluate by Correctness
-

In summary, the proposed intelligent self-healing analysis and decision-making framework constructs a graph-based representation for microservice systems by jointly modeling metrics, logs, and traces as multimodal monitoring data. On top of this representation, an LLM is used to semantically enhance node features and its anomaly priors are explicitly injected into the graph attention mechanism, enabling precise root cause identification. Finally, an LLM-based recovery agent with case retrieval and strategy generation completes the loop from anomaly perception and root cause localization to recovery strategy planning, providing a generalizable technical approach for reliability assurance in large-scale microservice-based business middle platforms.

4. Experiments and Results Analysis

This section evaluates GALR on reminding microservice observability data. We first introduce the datasets, fault injection process, baselines, and metrics. We then report results for (i) root cause localization and (ii) recovery strategy generation, followed by ablation studies to quantify the contribution of key components.

4.1. Datasets and Fault Injection

We evaluate the proposed framework on three representative microservice-system datasets, covering both realistic business scenarios and controllable fault-injection settings: Customer Service, Power Grid Resource, and SockShop. The Customer Service dataset simulates a user-facing business system with complex workflows and high-concurrency access patterns, and is used to assess root cause localization under service-dependency failures and performance degradation. The Power Grid Resource dataset represents a resource scheduling and monitoring scenario in the power systems domain, where monitoring data

are dominated by resource- and performance-related behaviors, posing stringent requirements on identifying temporal anomalies and resource-bottleneck faults. The SockShop dataset is collected locally by deploying the SockShop microservice benchmark on a Kubernetes cluster and injecting reproducible perturbations into specified services/instances using Chaos-Mesh while recording the fault type and injection time window to produce aligned labels. Locust is used to generate configurable concurrent requests and load curves to cover system states under varying access intensities.

All three datasets cover six typical fault types: network delay, network loss, CPU stress, memory stress, pod failure, and pod kill. Fault injection is conducted across different services and call paths, while each trace contains at most one injected fault and the fault corresponds to a single service instance. This single-node, single-fault setting ensures unambiguous supervision for root cause localization and enables consistent evaluation of recovery decision-making. The collected traces exhibit diverse dependency structures, with call-chain depths ranging from 5 to 20 hops.

Table 2 summarizes the key statistics of the three datasets. We split each dataset into training and test sets with a ratio of 6:4. During the split, the distribution of fault-injected nodes is constrained to be as balanced as possible between the training and test sets to mitigate overfitting to specific nodes or a single topology and to improve robustness. Although fault injection provides controllable and repeatable perturbations, it cannot fully cover the diversity of natural production incidents. Moreover, labels are aligned to analysis windows using the injection time as an anchor, which may not perfectly reflect delayed or cascading effects. Therefore, we use fault injection mainly to provide reproducible supervision and controlled comparisons, while acknowledging a potential gap to fully naturalistic production failures.

Table 2. Statistics of the experimental datasets.

Dataset	Traces	Services	Pods	Depth	Fault Types	Injected Faults
Customer Service	23,183	67	1–5	5–20	6	24
Power Grid Resource	19,872	89	1–5	5–20	6	32
SockShop	8981	15	1–5	5–20	6	30

4.2. Baselines and Evaluation Metrics

4.2.1. Baseline Models

To comprehensively evaluate the performance of the proposed intelligent root cause localization and recovery decision framework, we select two categories of representative baselines. The first category consists of basic graph neural network (GNN) models, including GCN [2], GraphSAGE [4], which mainly rely on the service call graph topology and node features. These baselines are used to assess the fundamental capability of graph-structured modeling for root cause localization. The second category comprises advanced state-of-the-art root cause localization methods, including DiagFusion [44], MicroRank [26], MicorEGRCL [45], and PDiagnose [46]. These methods exhibit strong performance in multimodal feature integration, graph representation learning, and probabilistic reasoning, and thus serve as important references for assessing the effectiveness of the LLM-based semantic enhancement and attention bias mechanisms in our framework.

4.2.2. Evaluation Metrics

The experimental evaluation focuses on two core dimensions: root cause localization and recovery strategy generation.

For root cause localization, we adopt ranking-based metrics to reflect realistic troubleshooting scenarios. Specifically, we report Top-*k* accuracy (Top-1, Top-3, and Top-10) to

evaluate whether the true root cause node is ranked among the top k highest-confidence candidates. In addition, Mean Reciprocal Rank (MRR) is used to measure the average ranking position of the first correctly identified root cause, jointly characterizing localization accuracy and troubleshooting efficiency.

For recovery strategy generation, we evaluate whether the generated recovery plan captures the key remediation actions defined by domain experts. Given the action set extracted from a generated plan and the corresponding standard action set in the recovery case library, we compute an overlap-based consistency score (defined in Section 3.4). A recovery strategy is considered correct if it achieves sufficient coverage of the standard actions. This metric provides a lightweight and reproducible proxy for assessing action-level validity, i.e., whether the generated plan includes the essential recovery operations. We do not explicitly evaluate environment-specific parameter values, as such validation typically requires online execution and system-level feedback.

4.3. Experimental Settings and Implementation Details

We build a controllable evaluation environment for fault diagnosis and root cause localization based on the SockShop microservice application deployed on a Kubernetes cluster. Multimodal observability data are automatically collected during runtime via standard monitoring components. On the metrics side, Prometheus periodically scrapes and stores key service-level indicators, including CPU usage, memory usage, request latency, and error-related signals. On the logs side, Fluent Bit is deployed as a DaemonSet across all cluster nodes to continuously collect logs from container standard output and node-level container log directories, while attaching Kubernetes metadata. The collected logs are structured into a unified format to support time-window alignment with metrics, cross-modal association, and offline modeling. To ensure cross-modal consistency, we use the fault injection time as an anchor to perform unified time slicing for both logs and metrics.

The baseline graph neural network adopts a graph attention network (GAT) with two GATConv layers and 4-head multi-head attention. All GNN models are trained using the AdamW optimizer with an initial learning rate of 0.001, a batch size of 32, and 30 epochs. Gradient clipping is applied with the gradient norm capped at 2.0 to stabilize training. Since fault nodes constitute only a small fraction of nodes in practical microservice graphs, we employ a weighted BCEWithLogitsLoss, where the positive-class weight is determined by the ratio of positive and negative samples in the training set to mitigate class imbalance and improve recall for anomalous nodes.

For semantic enhancement and auxiliary diagnosis, we employ DeepSeek V3 to analyze representative log snippets and summarize abnormal behaviors. The LLM module takes as input candidate root-cause nodes, anomalous subgraph summaries, and key log snippets within the corresponding time window, and outputs interpretable descriptions of fault causes to improve the readability and actionability of diagnostic results.

To support recovery suggestion generation, we construct a recovery case library $\mathcal{C}$ containing 500 curated recovery cases. Each case includes a fault type, fault description, fault-related features, and a corresponding set of standard recovery actions. For a localized root cause, similar cases are retrieved by encoding fault descriptions into a shared embedding space using the BGE-M3 embedding model and computing cosine similarity. The top- k retrieved cases are used as contextual prompts together with the current system status to guide the LLM in generating structured recovery plans.

The recovery actions are restricted to a compact and standardized action space commonly used in microservice operations, such as restart, rollback, migration, and scaling. Since all datasets adopt a single-node, single-fault injection setting and the recovery ac-

tions correspond to atomic operational procedures without strict order dependencies, we evaluate recovery strategies at the action level using the overlap-based consistency metric defined in Equation (13).

Root Cause Localization Results

Table 3 reports the root cause localization performance of GALR and all baselines on the three datasets in terms of Top- k accuracy and MRR. Overall, GALR achieves the best results on all datasets and across all metrics. The advantage is particularly pronounced for Top-1 accuracy, which reflects single-step troubleshooting efficiency, and for MRR, which summarizes the overall ranking quality. Averaged over the three datasets, GALR reaches an MRR of 0.931. By comparison, the strongest baseline (PDiagnose) attains an average MRR of about 0.902, so GALR yields a relative improvement of approximately 3.2%.

From the perspective of method categories, MicroRank and GCN mainly rely on static topology or simple aggregation, and their performance drops more clearly on complex scenarios with deeper dependency chains and richer business logic. As modeling capacity increases, MicorEGRCL and PDiagnose improve by incorporating multimodal fusion or more advanced graph learning and probabilistic reasoning. Nevertheless, GALR still consistently outperforms these methods. This suggests that, beyond graph-structured modeling alone, explicitly introducing LLM-derived semantic priors and amplifying them through attention bias can further improve the identification of diagnostically meaningful propagation paths and true root causes.

Figure 2 presents the cross-dataset averages of Top-1 accuracy and MRR for all methods. GALR consistently outperforms all baselines on both metrics, indicating that it maintains stable performance across different microservice environments and fault patterns. The detailed contribution of key components is further quantified in the ablation study.

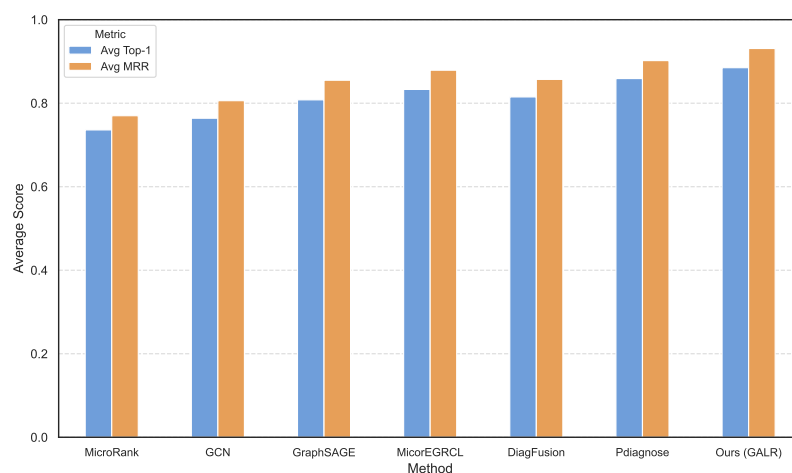


Figure 2. Average root cause localization performance comparison across three datasets (Avg Top-1 and Avg MRR).

Table 3. Comparison of root cause localization performance: Top- k accuracy and Mean Reciprocal Rank.

Method	Customer Service				Power Grid Resource				SockShop			
	Top-1	Top-3	Top-10	MRR	Top-1	Top-3	Top-10	MRR	Top-1	Top-3	Top-10	MRR
MicroRank	0.675	0.730	0.771	0.741	0.709	0.751	0.796	0.734	0.824	0.855	0.869	0.835
GCN	0.710	0.768	0.800	0.785	0.737	0.789	0.827	0.775	0.845	0.879	0.895	0.859
GraphSAGE	0.758	0.817	0.861	0.840	0.796	0.838	0.887	0.831	0.871	0.909	0.927	0.893
MicorEGRCL	0.784	0.842	0.881	0.865	0.817	0.860	0.901	0.852	0.899	0.938	0.954	0.919
DiagFusion	0.767	0.824	0.862	0.838	0.803	0.847	0.893	0.835	0.875	0.913	0.932	0.897
PDiagnose	0.811	0.865	0.908	0.887	0.849	0.887	0.932	0.881	0.916	0.957	0.972	0.938
GALR (Ours)	0.842	0.901	0.942	0.923	0.883	0.925	0.968	0.916	0.931	0.972	0.989	0.953

4.4. Recovery Strategy Generation Results

After root cause localization, we evaluate the retrieval-augmented LLM agent for recovery strategy generation on representative fault cases from SockShop, Power Grid Resource, and Customer Service. We compare three generation modes: pure LLM reasoning (zero-shot), few-shot prompting, and retrieval-augmented generation (LLM + RAG) within the GALR framework.

Table 4 summarizes the recovery strategy accuracy of the three modes. A consistent trend is observed: as more task-relevant information and domain experience are provided, the accuracy improves. The zero-shot setting performs worst across all datasets, especially on the more complex Customer Service dataset, indicating that purely relying on generic model knowledge is often insufficient for producing professional and executable recovery plans. Few-shot prompting improves results by providing a small number of generic examples, which helps the model better follow the task format and capture common remediation patterns.

Table 4. Comparison of recovery strategy generation accuracy (%).

Generation Method	SockShop	Power Grid Resource	Customer Service
Pure LLM (Zero-shot)	62.5	55.1	48.9
LLM + Few-shot	68.9	61.5	54.3
LLM + RAG (GALR)	79.2	75.8	70.1

In contrast, the LLM + RAG mode within GALR achieves the highest accuracy on all datasets. This improvement indicates that retrieving similar historical cases and supplying expert actions as contextual prompts can substantially enhance the specificity and feasibility of generated strategies. We evaluate recovery plans offline without executing them in a live system. Therefore, the reported strategy accuracy should be interpreted as action-level validity rather than guaranteed end-to-end operational success, which requires controlled deployment and online verification.

4.5. Ablation Study

To quantify the contributions of key components in GALR, we conduct ablation studies using Top-*k* accuracy and MRR. We focus on variants that remove the edge attention mechanism, replace the learnable attention with average-weighted aggregation, or use random aggregation. The ablation results are reported in Table 5.

Table 5. Ablation study of the root cause localization module in GALR (Top-*k* accuracy and MRR).

Variant	Customer Service				Power Grid Resource				SockShop			
	Top-1	Top-3	Top-10	MRR	Top-1	Top-3	Top-10	MRR	Top-1	Top-3	Top-10	MRR
w/o Edge Attention	0.771	0.825	0.865	0.835	0.803	0.845	0.888	0.828	0.854	0.895	0.918	0.875
Avg. Weighted Baseline	0.749	0.801	0.839	0.812	0.781	0.821	0.863	0.805	0.830	0.871	0.893	0.852
Random Aggregation	0.718	0.770	0.807	0.781	0.749	0.789	0.829	0.775	0.798	0.838	0.860	0.821
GALR (Full)	0.842	0.901	0.942	0.923	0.883	0.925	0.968	0.916	0.931	0.972	0.989	0.953

As shown in Table 5, removing the edge attention mechanism leads to consistent degradation in MRR across datasets, indicating that attention bias is important for highlighting diagnostically meaningful dependencies and suppressing irrelevant edges. Replacing GAT with average-weighted aggregation further decreases performance, suggesting that static weighting is insufficient to capture heterogeneous interactions in microservice call graphs. Random aggregation performs worst, which confirms that both graph structure and learnable attention are critical for effective localization. Overall, the ablation results support that

graph-structured attention and LLM-based semantic enhancement jointly contribute to the localization gains of GALR.

5. Discussion

The experimental results on three heterogeneous datasets suggest that GALR provides a practical way to couple dependency-aware graph reasoning with log-level semantic signals. The GNN component leverages topology and window-aligned metrics/traces for structured propagation modeling, while the LLM contributes a compact semantic prior from unstructured logs. This combination is particularly useful when anomalies are weak in a single modality: semantic cues help disambiguate candidates that are topologically plausible but operationally irrelevant, and graph propagation helps contextualize locally salient logs along dependency paths.

Despite these advantages, several limitations and deployment challenges remain and deserve a more balanced discussion. First, scalability may become a bottleneck in large systems. As the service graph grows, both neighborhood aggregation and LLM-side annotation/retrieval can increase latency, which may hinder near-real-time incident response. While the localization module can be deployed as an offline diagnostic service, production usage still requires careful budgeting of when to compute or refresh LLM-derived signals and how to amortize them across windows and repeated incidents.

Second, system integration is context-dependent. The effectiveness of GALR relies on the quality of multimodal alignment and the availability of stable logging/monitoring practices. Differences in log taxonomies, template stability, sampling policies, or missing observability signals can reduce the reliability of semantic priors and weaken cross-environment transfer. In this sense, the overall framework is general, but the achieved gains may vary with platform-specific conventions and data quality.

Third, recovery planning is the most safety-critical part. Even with retrieval grounding, LLMs may still produce incomplete or hallucinated strategies under distribution shifts (e.g., unseen failure patterns, noisy logs, or constraint changes). To reduce operational risk, our design separates generation from execution: the LLM proposes action types and parameters, and an additional parameter-extraction step maps them into predefined command templates under a restricted action space. Before any execution, parameter-level safety constraints should be enforced (e.g., allowlists for targets, bounded ranges, and disallowed operations), and unsafe plans should be rejected or escalated for manual confirmation. Accordingly, our current evaluation focuses on offline action-level validity, while full verification of parameter validity and online effectiveness requires controlled testing in real environments.

Finally, our experimental setting itself has limitations. Fault injection enables controllable and reproducible comparisons but does not fully cover the diversity and long-tail characteristics of production incidents, and window-level labels anchored at injection time may not perfectly capture delayed or cascading effects. In addition, the reported results are averaged over two runs under the same configuration, and we did not conduct statistical significance testing in this version. These aspects do not negate the observed improvements, but they indicate clear directions for more rigorous evaluation in future work.

6. Conclusions

This paper presented GALR, a unified framework that combines multimodal observability modeling, graph-based dependency reasoning, and LLM-assisted semantics for microservice root cause localization and recovery planning. By injecting LLM-derived semantic priors into graph attention and using retrieval-augmented prompting

for strategy generation, GALR provides an end-to-end pipeline from candidate ranking to actionable suggestions.

Experiments on three datasets show consistent improvements over representative baselines in Top- k /MRR for localization and in action-level strategy accuracy for recovery planning. However, the current results should be interpreted with several caveats. First, our evaluation relies on fault injection and window-aligned labels, which may not fully capture delayed or cascading effects in real incidents. Second, recovery plans are assessed offline using an action-overlap metric, without executing the suggested commands in a live system; thus, the reported accuracy reflects action validity rather than guaranteed operational success. Third, results are averaged over two runs under the same setup, and we do not report statistical significance in this version.

Future work will prioritize addressing these limitations. We will (i) improve scalability and latency via selective and incremental LLM annotation with caching and refresh policies, (ii) introduce stricter safety guardrails for recovery execution, including constrained action spaces and parameter-level validation, and (iii) establish controlled online verification together with more faithful evaluation protocols and benchmarks for recovery effectiveness. These directions are crucial to strengthen both the practicality and the reliability of LLM-augmented self-healing in real microservice environments.

Author Contributions: Conceptualization, W.Z. and Y.C.; Methodology, W.Z., Y.C. and R.C.; Software, W.Z., Z.Y. and L.Z.; Validation, W.Z., Y.C. and R.C.; Formal analysis, W.Z., F.P. and R.C.; Investigation, W.Z., F.P., L.Z. and Y.C.; Resources, W.Z., Z.Y., F.P., L.Z., Y.C. and R.C.; Data curation, F.P. and R.C.; Writing—original draft, W.Z.; Writing—review & editing, Z.Y., F.P., L.Z., Y.C. and R.C.; Visualization, F.P. and L.Z.; Project administration, W.Z., Z.Y. and R.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Information & Telecommunication Center of State Grid Corporation of China under the project “Research on Key Technologies for High-Reliability Middle-Platform Service Scheduling toward Integrated Whole-Network Management”, grant number SGSJ0000YWJS2400090.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: Author Wenya Zhan, Zhi Yang, Fang Peng, Le Zhang and Yiting Chen was employed by the company The Information & Telecommunication Center of STATE GRID Corporation of China. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Dragoni, N.; Giallorenzo, S.; Lluch Lafuente, A.; Mazzara, M.; Montesi, F.; Mustafin, R.; Safina, L. Microservices: Yesterday, Today, and Tomorrow. In *Present and Ulterior Software Engineering*; Springer: Berlin/Heidelberg, Germany, 2017; pp. 195–216.
2. Kipf, T.N.; Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In Proceedings of the International Conference on Learning Representations (ICLR), Toulon, France, 24–26 April 2017.
3. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; Bengio, Y. Graph Attention Networks. In Proceedings of the International Conference on Learning Representations (ICLR), Vancouver, BC, Canada, 30 April–3 May 2018.
4. Hamilton, W.L.; Ying, R.; Leskovec, J. Inductive Representation Learning on Large Graphs. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017.
5. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Neural Message Passing for Quantum Chemistry. In Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR, Sydney, Australia, 6–11 August 2017; pp. 1263–1272.
6. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Yu, P.S. A Comprehensive Survey on Graph Neural Networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *32*, 4–24. [[CrossRef](#)] [[PubMed](#)]
7. Zhou, X.; Peng, X.; Xie, T.; Sun, J.; Ji, C.; Li, W.; Ding, D. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Softw. Eng.* **2018**, *47*, 243–260. [[CrossRef](#)]

8. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017.
9. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models Are Few-Shot Learners. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Vancouver, BC, Canada, 6–12 December 2020.
10. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training Language Models to Follow Instructions with Human Feedback. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Vancouver, BC, Canada, 6–12 December 2020.
11. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR), Kigali, Rwanda, 1–5 May 2023.
12. Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv* **2023**, arXiv:2302.04761. [[CrossRef](#)]
13. Roy, D.; Zhang, X.; Bhave, R.; Bansal, C.; Las-Casas, P.; Fonseca, R.; Rajmohan, S. Exploring LLM-Based Agents for Root Cause Analysis. In Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) Companion, Porto de Galinhas, Brazil, 15–19 July 2024; pp. 208–219.
14. Ahmed, T.; Ghosh, S.; Bansal, C.; Zimmermann, T.; Zhang, X.; Rajmohan, S. Recommending Root-Cause and Mitigation Steps for Cloud Incidents Using LLMs. In Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; pp. 1737–1749.
15. Chen, Y.; Xie, H.; Ma, M.; Kang, Y.; Gao, X.; Shi, L.; Cao, Y.; Gao, X.; Fan, H.; Wen, M.; et al. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In Proceedings of the European Conference on Computer Systems (EuroSys), Athens, Greece, 22–25 April 2024; pp. 674–688.
16. Zhang, X.; Ghosh, S.; Bansal, C.; Wang, R.; Ma, M.; Kang, Y.; Rajmohan, S. Automated Root Causing of Cloud Incidents Using In-Context Learning with GPT-4. *arXiv* **2024**, arXiv:2401.13810. [[CrossRef](#)]
17. Wu, L.; Tordsson, J.; Elmroth, E.; Kao, O. Microrca: Root Cause Localization of Performance Issues in Microservices. In Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS), Budapest, Hungary, 20–24 April 2020; pp. 1–9.
18. Weng, J.; Wang, J.H.; Yang, J.; Yang, Y. Root Cause Analysis of Anomalies of Multitier Services in Public Clouds. *IEEE/ACM Trans. Netw.* **2018**, *26*, 1646–1659. [[CrossRef](#)]
19. Ma, M.; Lin, W.; Pan, D.; Wang, P. MS-Rank: Multi-Metric and Self-Adaptive Root Cause Diagnosis for Microservice Applications. In Proceedings of the IEEE International Conference on Web Services (ICWS), Milan, Italy, 8–13 July 2019; pp. 60–67.
20. Ma, M.; Lin, W.; Pan, D.; Wang, P. ServiceRank: Root Cause Identification of Anomaly in Large-Scale Microservice Architectures. *IEEE Trans. Dependable Secur. Comput.* **2021**, *19*, 3087–3100. [[CrossRef](#)]
21. Wang, P.; Xu, J.; Ma, M.; Lin, W.; Pan, D.; Wang, Y.; Chen, P. CloudRanger: Root Cause Identification for Cloud Native Systems. In Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), Washington, DC, USA, 1–4 May 2018; pp. 492–502.
22. Nguyen, H.; Shen, Z.; Tan, Y.; Gu, X. FChain: Toward Black-Box Online Fault Localization for Cloud Systems. In Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS), Philadelphia, PA, USA, 8–11 July 2013; pp. 21–30.
23. Liu, P.; Chen, Y.; Nie, X.; Zhu, J.; Zhang, S.; Sui, K.; Zhang, M.; Pei, D. FluxRank: A Widely-Deployable Framework to Automatically Localizing Root Cause Machines. In Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE), Berlin, Germany, 28–31 October 2019; pp. 35–46.
24. Lin, J.; Chen, P.; Zheng, Z. Microscope: Pinpoint Performance Issues with Causal Graphs in Micro-Service Environments. In Proceedings of the International Conference on Service-Oriented Computing (ICSOC), Hangzhou, China, 12 November 2018; pp. 3–20.
25. Ma, M.; Xu, J.; Wang, Y.; Chen, P.; Zhang, Z.; Wang, P. Automap: Diagnose Your Microservice-Based Web Applications Automatically. In Proceedings of the Web Conference (WWW), Taipei, Taiwan, 20–24 April 2020; pp. 246–258.
26. Yu, G.; Chen, P.; Chen, H.; Guan, Z.; Huang, Z.; Jing, L.; Weng, T.; Sun, X.; Li, X. Microrank: End-to-End Latency Issue Localization with Extended Spectrum Analysis. In Proceedings of the Web Conference (WWW), Ljubljana, Slovenia, 19–23 April 2021; pp. 3087–3098.
27. Li, Y.; Yu, G.; Chen, P.; Zhang, C.; Zheng, Z. MicroSketch: Lightweight and Adaptive Sketch Based Detection and Localization in Microservice Systems. In Proceedings of the International Conference on Service-Oriented Computing (ICSOC), Seville, Spain, 29 November–2 December 2022; pp. 219–236.
28. Yu, G.; Huang, Z.; Chen, P. TraceRank: Abnormal Service Localization with Dis-Aggregated End-to-End Tracing Data. *J. Softw. Evol. Process* **2023**, *35*, e2413. [[CrossRef](#)]

29. Wang, D.; Chen, Z.; Ni, J.; Tong, L.; Wang, Z.; Fu, Y.; Chen, H. Interdependent Causal Networks for Root Cause Localization. In Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), Long Beach, CA, USA, 6–10 August 2023; pp. 5051–5060.
30. Wang, D.; Chen, Z.; Fu, Y.; Liu, Y.; Chen, H. Incremental Causal Graph Learning for Online Root Cause Analysis. In Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), Long Beach, CA, USA, 6–10 August 2023; pp. 2269–2278.
31. Lu, X.; Xie, Z.; Li, Z.; Li, M.; Nie, X.; Zhao, N.; Yu, Q.; Zhang, S.; Sui, K.; Zhu, L.; et al. Generic and Robust Performance Diagnosis via Causal Inference for OLTP Database Systems. In Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Taormina, Italy, 16–19 May 2022; pp. 655–664.
32. Meng, Y.; Zhang, S.; Sun, Y.; Zhang, R.; Hu, Z.; Zhang, Y.; Jia, C.; Wang, Z.; Pei, D. Localizing Failure Root Causes in a Microservice Through Causality Inference. In Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS), Hangzhou, China, 15–17 June 2020; pp. 1–10.
33. Lee, C.; Yang, T.; Chen, Z.; Su, Y.; Lyu, M.R. EADRO: An End-to-End Troubleshooting Framework for Microservices on Multi-Source Data. In Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; pp. 1750–1762.
34. Gu, S.; Rong, G.; Ren, T.; Zhang, H.; Shen, H.; Yu, Y.; Li, X.; Ouyang, J.; Chen, C. TrinityRCL: Multi-Granular and Code-Level Root Cause Localization Using Multiple Types of Telemetry Data. *IEEE Trans. Softw. Eng.* **2023**, *49*, 3071–3088. [\[CrossRef\]](#)
35. Sui, Y.; Zhang, Y.; Sun, J.; Xu, T.; Zhang, S.; Li, Z.; Sun, Y.; Guo, F.; Shen, J.; Zhang, Y.; et al. LogKG: Log Failure Diagnosis Through Knowledge Graph. *IEEE Trans. Serv. Comput.* **2023**, *16*, 3493–3507. [\[CrossRef\]](#)
36. Li, X.; Chen, P.; Jing, L.; He, Z.; Yu, G. SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. *IEEE Trans. Dependable Secur. Comput.* **2022**, *20*, 2762–2780. [\[CrossRef\]](#)
37. Yuan, Y.; Shi, W.; Liang, B.; Qin, B. An Approach to Cloud Execution Failure Diagnosis Based on Exception Logs in OpenStack. In Proceedings of the IEEE International Conference on Cloud Computing (CLOUD), Milan, Italy, 8–13 July 2019; pp. 124–131.
38. Amar, A.; Rigby, P.C. Mining Historical Test Logs to Predict Bugs and Localize Faults in the Test Logs. In Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; pp. 140–151.
39. Jia, T.; Chen, P.; Yang, L.; Li, Y.; Meng, F.; Xu, J. An Approach for Anomaly Diagnosis Based on Hybrid Graph Model with Logs for Distributed Services. In Proceedings of the IEEE International Conference on Web Services (ICWS), Honolulu, HI, USA, 25–30 June 2017; pp. 25–32.
40. Wang, L.; Zhang, C.; Ding, R.; Xu, Y.; Chen, Q.; Zou, W.; Chen, Q.; Zhang, M.; Gao, X.; Fan, H. Root Cause Analysis for Microservice Systems via Hierarchical Reinforcement Learning from Human Feedback. In Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), Long Beach, CA, USA, 6–10 August 2023; pp. 5116–5125.
41. Rouf, R.; Rasoloveicy, M.; Litoiu, M.; Nagar, S.; Mohapatra, P.; Gupta, P.; Watts, I. InstantOps: Joint Failure Prediction and Root Cause Identification in Microservices. In Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE), London, UK, 7–11 May 2024; pp. 119–129.
42. Wu, L.; Bogatinovski, J.; Nedelkoski, S.; Tordsson, J.; Kao, O. Performance Diagnosis in Cloud Microservices Using Deep Learning. In Proceedings of the International Conference on Service-Oriented Computing (ICSOC), Dubai, United Arab Emirates, 14–17 December 2020; pp. 85–96.
43. Yu, G.; Chen, P.; He, Z.; Yan, Q.; Luo, Y.; Li, F.; Zheng, Z. ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems. In Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Porto de Galinhas, Brazil, 15–19 July 2024; pp. 24–46.
44. Zhang, S.; Jin, P.; Lin, Z.; Sun, Y.; Zhang, B.; Xia, S.; Li, Z.; Zhong, Z.; Ma, M.; Jin, W.; et al. Robust Failure Diagnosis of Microservice System Through Multimodal Data. *IEEE Trans. Serv. Comput.* **2023**, *16*, 3851–3864. [\[CrossRef\]](#)
45. Chen, R.; Ren, J.; Wang, L.; Pu, Y.; Yang, K.; Wu, W. MicroEGRCL: An Edge-Attention-Based Graph Neural Network Approach for Root Cause Localization in Microservice Systems. In *Service-Oriented Computing, Proceedings of the 20th International Conference, ICSOC 2022, Seville, Spain, 29 November–2 December 2022*; Troya, J., Medjahed, B., Piattini, M., Yao, L., Fernández, P., Ruiz-Cortés, A., Eds.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2022; Volume 13740, pp. 264–272.
46. Hou, C.; Jia, T.; Wu, Y.; Li, Y.; Han, J. Diagnosing Performance Issues in Microservices with Heterogeneous Data Source. In Proceedings of the 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom), New York, NY, USA, 30 September–3 October 2021; IEEE: New York, NY, USA, 2021; pp. 493–500.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.